

Memoria UT4 Centrada En La Tarea

Explicación directa de cómo se cumplen los apartados pedidos en el proyecto de pruebas automatizadas de la Liga Deportiva.

Asignatura: Optativa: Frameworks

Curso: 2º DAM - 2025/2026

Módulo principal: Jugadores

Tecnologías: Angular, Laravel, Jasmine/Karma, PHPUnit, Cypress y GitHub Actions

1. Pruebas unitarias en Angular

La tarea pide desarrollar pruebas unitarias en Angular con Jasmine/Karma para un componente o servicio del módulo de jugadores. Esto se cumple probando el servicio `JugadorService` y el componente de jugadores.

- `src/app/ligaDeportiva/services/jugador.service.spec.ts`
- `src/app/ligaDeportiva/components/jugadores.component/jugadores.component.spec.ts`

En estas pruebas se valida que el listado inicial se carga correctamente, que se pueden crear jugadores, que se pueden actualizar y eliminar, y que aparece un error cuando se intenta modificar un jugador que no existe.

2. Pruebas unitarias en Laravel

La tarea pide comprobar la funcionalidad de modelos y controladores relacionados con jugadores. En Laravel se han creado pruebas unitarias para el modelo `Jugador` y para un servicio que simula estadísticas externas.

- `tests/Unit/JugadorModelTest.php`
- `tests/Unit/JugadorEstadisticasClientTest.php`

Se comprueba que el modelo usa la tabla correcta, que tiene los campos asignables esperados, que puede crearse con datos válidos y que mantiene la relación con el club.

3. Lógica básica de negocio

La lógica de negocio principal se centra en que los jugadores estén bien relacionados con sus clubes y en que los datos enviados sean válidos.

- Un jugador debe tener nombre, posición, dorsal y club.
- El dorsal debe estar entre 1 y 99.
- No puede repetirse el mismo dorsal dentro del mismo club.
- Sí se permite el mismo dorsal si pertenece a clubes distintos.
- Crear, editar y eliminar jugadores requiere permisos de administrador.

4. Pruebas Feature en Laravel para /api/jugadores

La tarea pide pruebas de integración o Feature Tests para verificar el endpoint `/api/jugadores`, validando JSON, códigos de estado e interacción con base de datos. Esto se cubre con los tests del controlador y del flujo completo del módulo.

- `tests/Feature/JugadorControllerTest.php`
- `tests/Feature/JugadorModuleWorkflowTest.php`
- `tests/Feature/AngularJugadorContractTest.php`

Estas pruebas hacen peticiones reales contra la aplicación Laravel en entorno de testing. Se comprueba que el listado responde con estado 200, que la creación responde con 201, que los errores de validación responden con 422 y que los cambios se guardan o se eliminan realmente de la base de datos.

5. Integración en Angular con HTTP simulado

La tarea pide probar la comunicación entre componentes y servicios simulando peticiones HTTP mediante `HttpClientTestingModule`. Esto se realiza en:

- `src/app/ligaDeportiva/components/jugadores-http.integration.spec.ts`

El test crea un componente de lista y un servicio que llama a `/api/jugadores`. Después se intercepta la petición con `HttpTestingController` y se devuelve una respuesta JSON falsa. También se simula un error HTTP 500 para comprobar que el componente guarda un mensaje de error controlado.

6. Pruebas E2E con Cypress

La tarea pide simular un flujo real de usuario, incluyendo un caso exitoso y otro de error controlado. Esto se cumple con Cypress en:

- `cypress/e2e/jugadores.cy.ts`

Los casos implementados son:

- Un usuario entra en la página pública de jugadores y ve el listado.
- Un administrador intenta iniciar sesión con contraseña incorrecta y aparece un error.
- Un administrador inicia sesión, crea un jugador, lo edita y comprueba que aparece actualizado.

7. Mocks y simulación de llamadas HTTP

La tarea pide crear datos simulados y evitar depender de una base de datos o servidor real durante las pruebas. Se ha hecho de varias formas:

Apartado pedido	Cómo se cumple
Mock de jugadores en JSON	Cypress incluye fixtures en <code>cypress/fixtures</code> , como <code>jugadores-publicos.json</code> y <code>jugadores-admin.json</code> .
Angular con respuestas falsas	Se usa <code>HttpTestingController</code> para interceptar <code>/api/jugadores</code> y devolver datos controlados.
Laravel con datos controlados	Se usan factories como <code>JugadorFactory</code> y <code>ClubFactory</code> para preparar datos de prueba.
Laravel con HTTP falso	Se usa <code>Http::fake()</code> en <code>JugadorEstadisticasClientTest.php</code> para simular una respuesta externa.

8. Integración continua CI/CD

La tarea pide un pipeline de GitHub Actions que ejecute automáticamente las pruebas de Angular, Laravel y Cypress en cada push o pull request a la rama principal. El proyecto tiene workflows separados para frontend y backend.

- Angular: `ligaDeportiva2/.github/workflows/tests.yml`
- Laravel: `ligaDeportiva-backend/.github/workflows/tests.yml`

En Angular el workflow instala dependencias, ejecuta Jasmine/Karma, arranca la aplicación, ejecuta Cypress y compila. En Laravel instala Composer, comprueba estilo con Pint y ejecuta Unit Tests y Feature Tests.

9. Resultados comprobados

Comando	Resultado
<code>npm run test:ci</code>	17 pruebas Angular superadas.
<code>npm run test:e2e</code>	3 pruebas Cypress superadas.
<code>npm run build</code>	Compilación Angular correcta.
<code>php artisan test</code>	41 pruebas Laravel y 179 aserciones superadas.
<code>vendor/bin/pint --test</code>	Formato PHP correcto.

10. Conclusión

La implementación cumple los puntos técnicos pedidos en la tarea: pruebas unitarias en Angular y Laravel, Feature Tests del endpoint `/api/jugadores` , integración Angular con HTTP simulado, pruebas E2E con Cypress, mocks y CI/CD con GitHub Actions. Lo único que queda fuera del código es preparar el enlace desplegado en un archivo `.txt` y comprimir el proyecto para la entrega.